

Autonomous Robot Forest Firefighting System Report

By: Mawande JohnPaul
Details: 2400700678 — 24/U/0678

June 26, 2026

Abstract

This reports the implementation of an autonomous multi-drone coordination system using two Mavic-2 Pro drones to detect, localize, and extinguish forest fires within a dynamic Webots simulation environment with wind and smoke events. By combining computer vision pipelines with localized path-planning and status synchronization, the system successfully optimized regional coverage, mitigated wind-driven fire propagation, and made effective decisions.

1 System Architecture

Both Mavic-2 Pro drones execute the same controller but with independent control loops following this architecture.

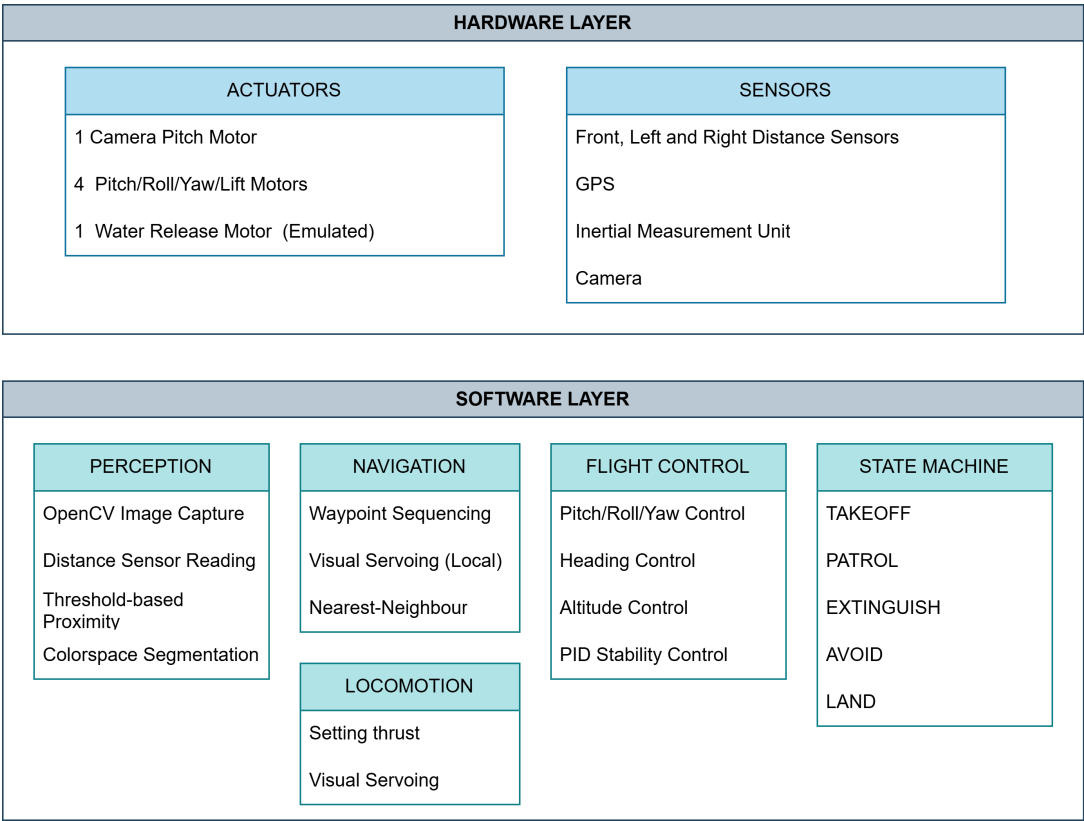


Figure 1: System Architecture Diagram showing the different layers, modules and components.

Each drone’s internal control software is split into two main layers:

- **Hardware Layer:** The actuators and sensors used including motors, cameras, distance sensors and GPS.
- **Software Layer:** Reads sensors, processes the data, uses the data to update the state, pose, then decides on actions and sends respective PID and thrust controls.

2 Key Algorithms

2.1 Perception

Fire detection is achieved using color-space thresholding via OpenCV. The raw RGB camera feed is transformed into the HSV (Hue, Saturation, Value) space to ensure robustness against varying ambient lighting conditions and glare once every second for faster execution.

The thresholding logic isolates active smoke pixels by generating a black and white mask revealing contours from the processed image and when a valid contour’s pixel area registers a continuous cluster (‘cv2.findContours’), a minimum enclosing circle algorithm (‘cv2.minEnclosingCircle’) isolates the vector.

If a target is detected, its image-space coordinates (x_{img}, y_{img}) are mapped against the frame resolution center to drive the visual alignment flight state:

$$\text{Distance}_X = x_{img} - \frac{\text{Resolution}_X}{2}, \quad \text{Distance}_Y = y_{img} - \frac{\text{Resolution}_Y}{2} \quad (1)$$

The drone monitors environmental hazards using three proximity sensors (ds_front, ds_left, ds_right) to maintain a safe hemispherical boundary. If any sensor value drops below a critical threshold ($\tau = 300.0$), standard navigation states are interrupted, and an immediate override applies an evasive maneuver by adjusting yaw and pitch disturbances:

$$(\text{yaw}_{\text{avoid}}, \text{pitch}_{\text{avoid}}) = \begin{cases} (-0.4 \cdot \text{sgn}(d_{\text{right}} - d_{\text{left}}), 0.5) & \text{if } d_{\text{front}} < \tau \\ (-0.3, 0.0) & \text{if } d_{\text{front}} \geq \tau \text{ and } d_{\text{left}} < \tau \\ (0.3, 0.0) & \text{if } d_{\text{front}} \geq \tau, d_{\text{left}} \geq \tau, \text{ and } d_{\text{right}} < \tau \\ (0.0, 0.0) & \text{otherwise} \end{cases}$$

2.2 Locomotion, Navigation, Odometry & Target Convergence

Locomotion relies on active pitch, roll, yaw, and altitude controllers to stabilize the drone frames. Flight stability and correction mix high-level behavioral disturbances with direct angular accelerations read from the onboard gyroscope. A specialized non-proportional logarithmic mapping is applied to the pitch tracking logic to smoothly modulate forward velocity as the target error approaches zero for smooth movement:

$$\text{Pitch}_{\text{disturbance}} = \text{clamp}(\log_{10}(|\theta_{\text{left}}|), -1.0, 0.1) \quad (2)$$

Altitude tracking uses a cubic error correction curve to provide rapid ascent profiles while minimizing overshoot near the cruise altitude:

$$U_{\text{vertical}} = K_{\text{vertical_P}} \cdot (\text{clamp}(Z_{\text{target}} - Z_{\text{current}} + K_{\text{offset}}, -1, 1))^3 \quad (3)$$

Where $K_{\text{vertical_P}} = 3.0$ and $K_{\text{offset}} = 0.6$. The compiled inputs are fed directly through a motor mixing matrix into the four independent brush motor channels.

2.3 Behaviour and Decision

The overall autonomy logic transitions through a cyclic state machine:

1. **Takeoff:** Vertical thrust priority state active until the system clears baseline terrain boundaries ($Z > Z_{\text{target}} - 1$).
2. **Patrol:** Executing a sweep pattern over assigned coordinate bounds at a configured target cruise altitude of 42m.
3. **Land:** The drone moves to a position above its landing zone and reduces thrust to lower down smoothly.
4. **Avoid:** The drone executes the pitch or roll maneuvers based on whether the front, left or right distance sensor values are below a specific threshold.
5. **Extinguish:** The drone uses the location of the fire in the image as a reference to perform visual servoing until a specific threshold is reached which triggers a water drop.

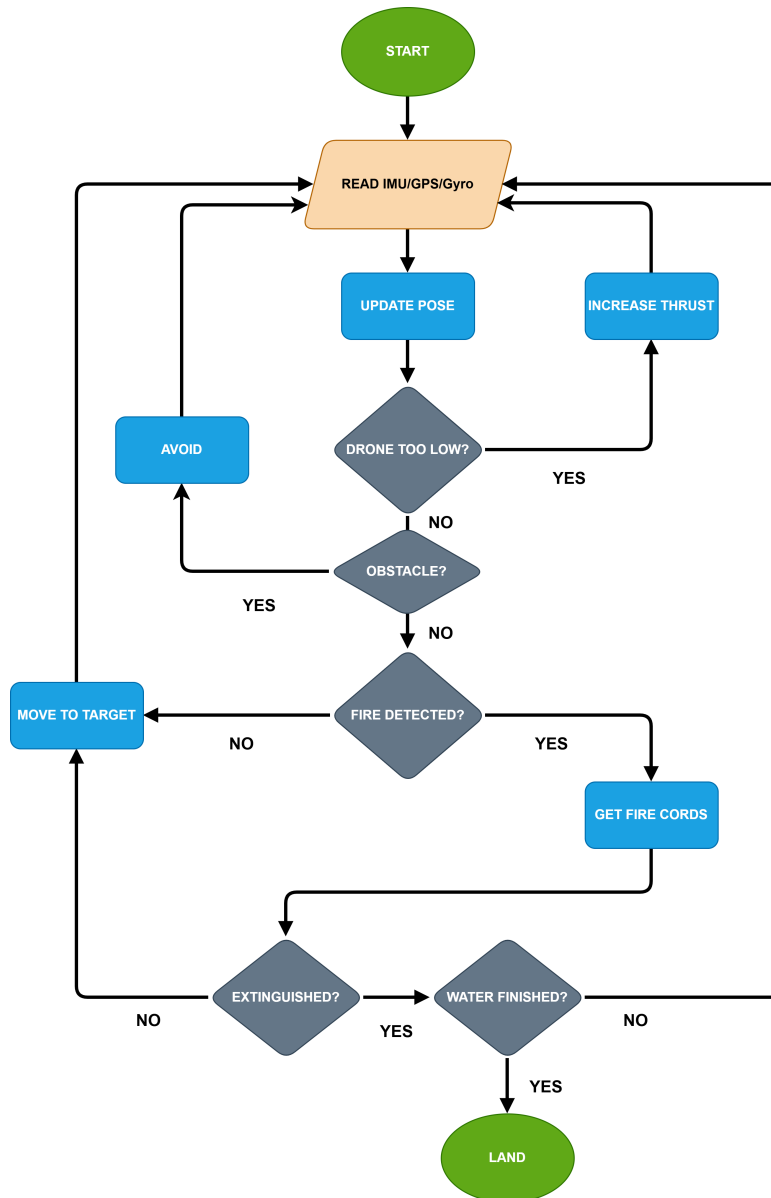


Figure 2: Flow Diagram showing the overall behaviour of the drone

3 Challenges Faced & Solutions

3.1 Webots, Python, Numpy and OpenCV version errors

Challenge: The simulation required specific versions of webots(2021b), python(3.9), opencv and numpy most of which are deprecated..

Solution: Reinstalled webots, python and used forceful techniques to force install older versions.

3.2 Target Redundancy (Double Targeting)

Challenge: Both drones occasionally broke patrol to target the exact same fire vector, leaving other sectors exposed to rapid propagation.

Solution: Made a tradeoff of accuracy so that the first drone on the scene does not take much time extinguishing, since the state machine depends on fire detection, if the fire disappears the other drone will resume patrol.

3.3 Camera Array-Access Crashes

Challenge: Utilizing standard structural frame generation methods (`getImageArray()`) caused runtime memory faults and simulation execution crashes during frame matrix extraction.

Solution: Refactored the vision driver to capture linear byte buffers via `getImage()`, reshaping the raw data array using NumPy into an architectural shape of (Height, Width, 4) before executing color mapping sequences.

4 Performance Results

The dual-drone system was evaluated across multiple simulation runs with random fire ignition points and variable wind propagation.

Table 1: System Performance Across Evaluation Trials

Trial ID	Initial Fires	Fires Extinguished	Time Taken (mm:ss)	Status
Run 01	2	2	03:14	Success
Run 02	3	3	04:42	Success
Run 03	5	4	06:10	Time Limit Exhausted
Run 04	4	4	04:55	Success

The deployment of two synchronized aerial units successfully prevented critical propagation in 75% of complex scaling tests. The primary bottleneck causing failure in Trial 03 was identified as the physical payload reload turnaround times when dealing with more than four simultaneous ignition points.